FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P2

Relatório consolidado

arquitetura e atividades do 2º bimestre

Avaliação individual · 2026.2

Felipe Takayuki Tanaka Takahashi

Produto: **Metastatistic**
https://github.com/Felipe-Takayuki/metastatistic-commercial

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 13/11/2026

Sumário

1 Introdução 2
2 Metodologia 2
3 Arquitetura do SaaS 2
3.1 Visão geral e componentes 2
3.2 Onde roda 4
3.3 Fluxo de uma requisição 4
3.4 Fluxo de assinatura 5
3.5 Modelo de dados do domínio 6
3.6 Segurança 6
3.7 Observabilidade, CI/CD e documentação da API 7
3.8 Camadas IaaS, PaaS e SaaS do projeto 7
4 Quadro de entregas 8
5 Atividades realizadas 8
5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe 8
5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve 10
5.3 Atividade 9 — Fork e plano de negócio 11
5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes 14
5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe 18
6 Considerações finais 22
7 Declaração de autoria 22

1 Introdução

A disciplina de Introdução à Computação em Nuvem (ISW055), do Prof. Allan Lincoln Rodrigues Siriani na Fatec Pompeia, ensina na prática como um sistema deixa de ser um programa e passa a ser um serviço: separado em componentes que conversam por rede, empacotado em contêineres, publicado em servidor, cobrado por assinatura e observado em produção. No 1º bimestre o fio condutor foi um catálogo de filmes que ganhou, semana a semana, autenticação, papéis, logs e upload. No 2º bimestre o projeto passou a ser um negócio de verdade.

O produto é o **Metastatistic**, um SaaS de Business Intelligence com estatística inferencial. A dor: bioestatísticos, pesquisadores clínicos e equipes de P&D gastam de 15 a 40 horas por semana limpando planilhas, rodando testes em programas de desktop caros (como o SPSS) e redigitando tabelas no Word para o padrão APA 7ª edição, sem trilha de auditoria. O Metastatistic junta a preparação visual de dados do BI com testes (t, ANOVA, ANCOVA, não paramétricos, bayesianos) e relatórios prontos, na nuvem, com papéis, auditoria e cobrança. Os planos são Free (R$ 0) e Pro (R$ 249,00 por mês).

Este relatório consolida o 2º bimestre. Ele traz a arquitetura do sistema como ela existe no repositório (componentes, onde roda, os fluxos de requisição e de assinatura, o modelo de dados, a segurança, a observabilidade e as camadas IaaS, PaaS e SaaS), o quadro de entregas das atividades 7 a 11 com data planejada e realizada, e uma ficha por atividade com evidência. As atividades que não foram entregues ou que foram entregues com atraso aparecem declaradas como tal, e as limitações do que foi feito estão escritas nas fichas e nas considerações finais.

2 Metodologia

As atividades 7 e 8 pertencem ao repositório público do catálogo de filmes (`atividade-filmes`), o mesmo do 1º bimestre. A partir da atividade 9 o trabalho passou ao repositório do produto, `metastatistic-commercial`, que reúne o código (front, API, gateway e microsserviços), a documentação em `docs/` e o backlog em issues. O relatório de cada atividade está em `docs/`.

As datas e horas vieram de duas fontes verificáveis: o horário de criação da issue de entrega no GitHub (que menciona o professor) e o histórico de commits (`git log`). Os horários estão no fuso de Brasília. Os prints de páginas do GitHub foram tirados por um script com Chromium (`docs/p2/prints/`), com o horário da captura escrito no rodapé de cada imagem e registrado em `manifesto-github.json`. Os prints do sistema vêm do script `apps/web/e2e/evidencias.mjs`, que sobe um navegador contra a pilha em execução, e seguem o mesmo padrão (rodapé e `docs/evidencias/manifesto.json`). Nenhuma imagem foi editada depois de capturada.

Uma ressalva de método: os prints do sistema das atividades 10 e 11 foram feitos executando a mesma pilha em uma máquina local, sem Docker, e com o Stripe em modo simulado (`STRIPE_MOCK_MODE=true`). Isso está dito em cada ficha e no painel admin do próprio sistema, que mostra “Stripe simulado (mock)”.

3 Arquitetura do SaaS

3.1 Visão geral e componentes

O navegador fala só com o **gateway**, o único contêiner que publica porta. O gateway valida a sessão, confere se o usuário é membro da organização pedida, descobre o plano dela e roteia a requisição para

o serviço certo, repassando identidade, organização, plano e cotas verificados. Todo o resto vive na rede interna do Docker.

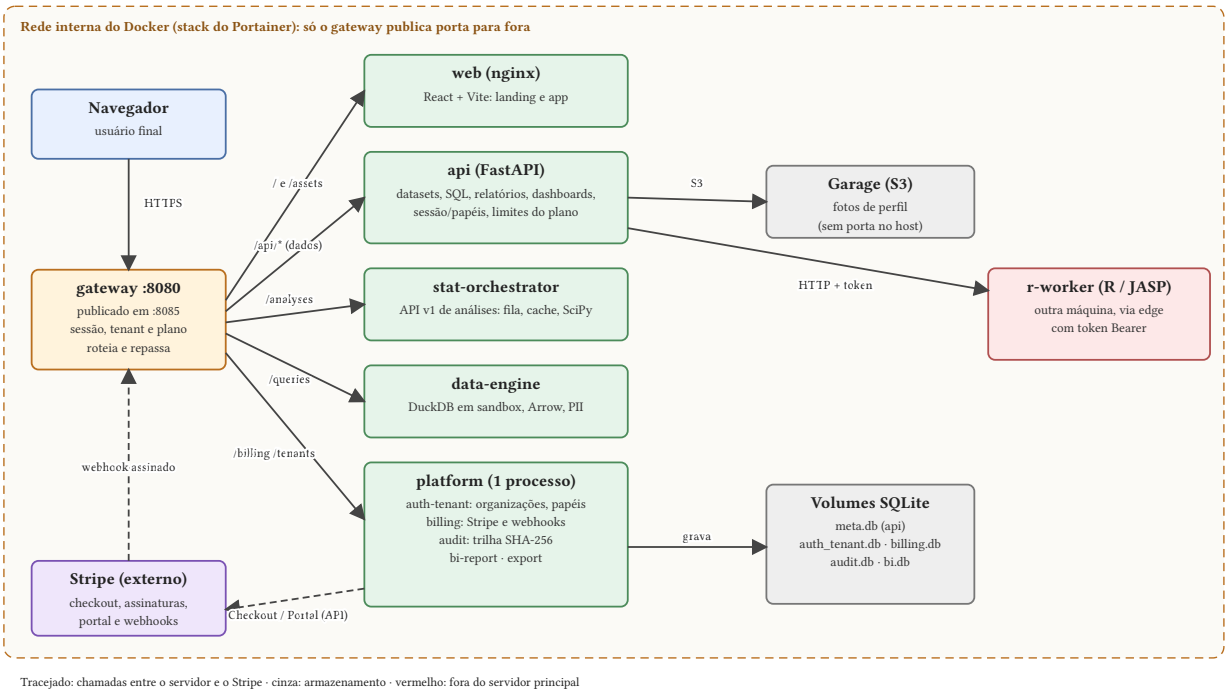


Figura 1: Diagrama de componentes e microsserviços do SaaS

Componente	Responsabilidade	Tecnologia / imagem	Público?
gateway	autenticação da sessão, autorização por organização, roteamento, plano e cotas repassados à API, saúde agregada	FastAPI + httpx	sim (única porta)
web	landing page pública, app (explorar, SQL, análises, relatórios, dashboards, planos, painel admin)	React 19 + Vite, nginx	via gateway
api	datasets (CSV para Parquet), SQL, transformações, análises, relatórios, dashboards, usuários e papéis, auditoria, limites do plano	FastAPI + DuckDB + SQLite	via gateway
platform	um processo com cinco serviços: auth-tenant (organizações, membros, papéis, convites, cotas), billing (Stripe), audit (trilha SHA-256), bi-report e export	FastAPI	via gateway (prefixos internos)
stat--orchestrator	API v1 de análises com fila priorizada e cache	FastAPI + SciPy	via gateway
data-engine	consultas em DuckDB isolado, Arrow, mascaramento de PII	FastAPI + DuckDB	via gateway
r-worker	motor R/JASP das análises, em outra máquina, acessado por um edge com token	R + JASP	não (token Bearer)

Componente	Responsabilidade	Tecnologia / imagem	Público?
Garage	fotos de perfil (objetos compatíveis com S3)	Garage	não
Stripe (externo)	checkout, assinaturas, portal do cliente, webhooks	serviço SaaS	webhook para o gateway

3.2 Onde roda

A stack é definida em `docker-compose.portainer.yml` (e `docker-compose.yml` para desenvolvimento). No Portainer ela é criada como stack do tipo Repository, apontando para o repositório do produto (`main`), porque o compose constrói as imagens a partir do código. Contêineres e tetos de memória: gateway 128 MB, platform 192 MB, stat-orchestrator 256 MB, data-engine 256 MB, api 512 MB, web 32 MB, garage 128 MB e garage-init 64 MB. O worker R fica em outra máquina para poupar até 1,5 GB do servidor principal.

As variáveis de ambiente (apenas os nomes; os valores ficam na stack, nunca no repositório): `JWT_SECRET_KEY`, `R_WORKER_URLS`, `R_WORKER_TOKEN`, `ENVIRONMENT`, `AUTH_ENABLED`, `COOKIE_SECURE`, `DATASETS_PRIVATE_BY_DEFAULT`, `STRIPE_MOCK_MODE`, `STRIPE_SECRET_KEY`, `STRIPE_PUBLISHABLE_KEY`, `STRIPE_WEBHOOK_SECRET`, `STRIPE_PRICE_ID`, `APP_URL`, `S3_ACCESS_KEY`, `S3_SECRET_KEY`, `GARAGE_RPC_SECRET` e `GARAGE_ADMIN_TOKEN`. O `.env.example` traz só os nomes.

URL pública. Não informo aqui uma URL no ar como prova: o endereço da atividade 7 (`felipe-takahashi-isw055.lapps.studio`) respondeu 502 em 13/11/2026, e os prints do sistema desta entrega foram feitos executando a pilha localmente. Com `ENVIRONMENT=production` o gateway, o auth-tenant e o billing se recusam a iniciar com segredo padrão, autorização desligada ou Stripe simulado.

3.3 Fluxo de uma requisição

Exemplo: a pesquisadora (papel `editor`) envia um CSV de dados de um estudo.

1. O navegador faz `POST /api/datasets` com o cookie de sessão e o cabeçalho `X-Tenant-ID` da organização ativa.
2. O gateway pergunta à api quem é o dono da sessão (`/auth/me`: usuário e papel da plataforma).
3. O gateway pergunta ao auth-tenant se esse usuário é membro da organização e com que papel; recebe o plano e as cotas. Os cabeçalhos `X-Tenant-*` que o cliente tenha mandado são descartados e substituídos pelos verificados.
4. A api confere o papel (`editor` ou mais) e o limite do plano: número de datasets da organização e tamanho do arquivo. Acima do limite, responde 402 com a mensagem do limite e como liberar.
5. Dentro do limite, o DuckDB lê o CSV, infere os tipos, grava o Parquet e registra o dataset no `meta.db`, com a organização e o dono. O dataset nasce privado ao dono.
6. A api grava o evento: linha na tabela `audit` e uma linha JSON no log do processo (`dataset.create`).
7. A resposta volta pelo gateway ao navegador (201 e o dataset).

3.4 Fluxo de assinatura

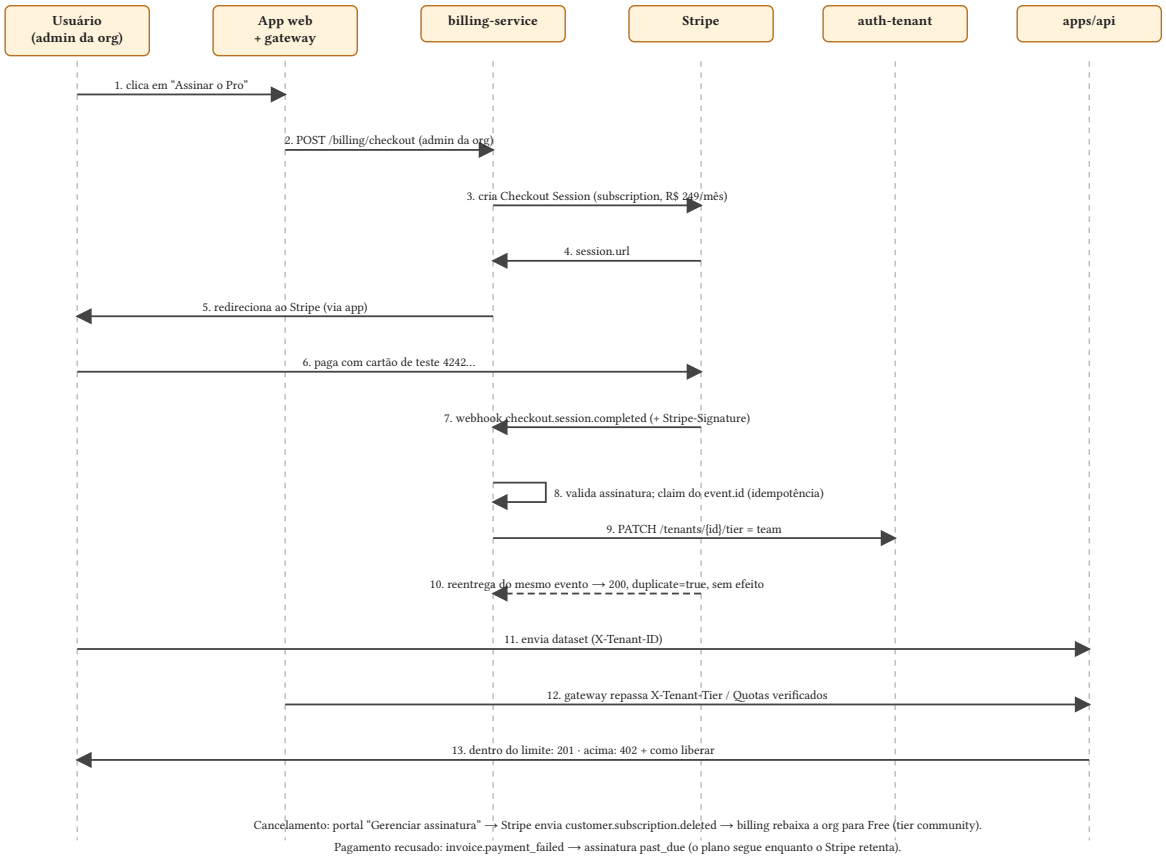


Figura 2: Fluxo de assinatura: checkout, webhook, plano no banco e limites

O administrador da organização clica em "Assinar o Pro". O billing-service cria uma Checkout Session em modo `subscription` com o preço mensal em reais e devolve a URL do Stripe, e o pagamento acontece na página do Stripe, nunca no sistema. A ativação do plano não depende do redirecionamento: ela vem do webhook. O Stripe chama `POST /api/billing/webhook`; o billing valida a assinatura do cabeçalho `Stripe-Signature` com o segredo do webhook (sem assinatura válida a resposta é 400 e nada é gravado) e trata quatro eventos: `checkout.session.completed` (ativa o Pro e promove a organização no auth-tenant), `customer.subscription.updated` (status, período e cancelamento no fim do ciclo), `customer.subscription.deleted` (rebaixa a organização para Free) e `invoice.payment_failed` (fatura em aberto e assinatura `past_due`).

Evento repetido. O Stripe reentrega eventos. O `event.id` é a chave primária da tabela `webhook_events`; o evento é reivindicado antes de qualquer efeito. A segunda entrega responde 200 com `duplicate: true` e não repete o efeito (a promoção da organização acontece uma vez só). Se o processamento falha, o evento é marcado `failed`, a resposta é 500 e a retentativa do Stripe consegue processá-lo. O cancelamento é feito pelo portal do cliente ("Gerenciar assinatura"), que gera `customer.subscription.updated` e, ao fim do ciclo, `customer.subscription.deleted`. A partir da assinatura, o gateway passa a repassar o plano novo e os limites mudam na requisição seguinte.

3.5 Modelo de dados do domínio

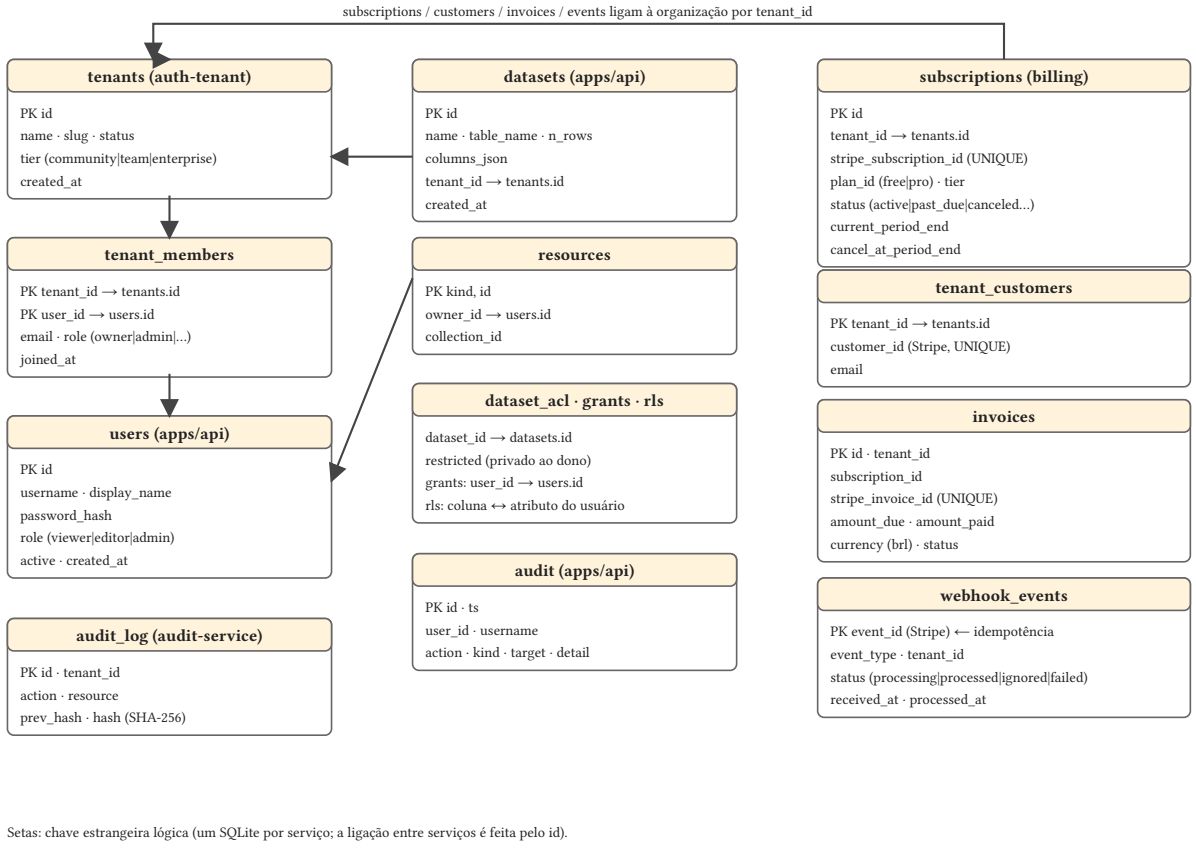


Figura 3: Modelo de dados do domínio (um SQLite por serviço)

Cada serviço tem o próprio SQLite, em volume. As organizações e seus membros ficam no auth--tenant; o `tier` da organização (`community` é o Free, `team` é o Pro) é o que define as cotas. A assinatura do Stripe fica no billing (`subscriptions.stripe_subscription_id`, `tenant_customers.customer_id`), ligada à organização por `tenant_id`. Usuários, datasets, relatórios, dashboards e a auditoria do app ficam na api; `resources.owner_id` guarda o dono de cada item e `dataset_acl`, `dataset_grants` e `dataset_rls` decidem quem lê cada dataset. A ligação entre serviços é pelo identificador, não por chave estrangeira do banco.

3.6 Segurança

Há dois níveis de papel. Na plataforma (api): `viewer`, `editor` e `admin`. Na organização (auth-tenant): `owner`, `admin`, `statistician`, `analyst` e `viewer`. Assinar o plano e abrir o portal exigem ser `owner` ou `admin` da organização.

Ação (papel da plataforma)	admin	editor	viewer
enviar dataset, criar relatório e dashboard	sim	sim	não
ver um dataset privado de outro usuário	sim	não	não
editar item de outro usuário	sim	não	não
gerenciar usuários e ver a auditoria	sim	não	não
ver o painel de assinaturas (todas as organizações)	sim	não	não
ler o que o dono compartilhou ou deixou aberto	sim	sim	sim

Isolamento. Cada usuário vê só os próprios dados: o dataset novo nasce restrito ao dono (`DATASETS_PRIVATE_BY_DEFAULT`); outro usuário não o vê na lista nem no SQL e recebe 404 ao abrir, sem confirmar que ele existe. O dono compartilha em "Acesso…", e uma regra de linha pode filtrar por atributo do usuário. A limitação conhecida está no backlog: a separação é por usuário; os dados da api ainda não são particionados por organização (issue #2). Outras proteções: o gateway recusa criar organização com plano pago (senão alguém criaria uma Pro sem pagar), endpoints internos (troca de tier, estatísticas) não passam pelo gateway e o painel de assinaturas devolve 403 a quem não é administrador da plataforma.

Segredos. Ficam em variáveis de ambiente da stack. O `.env` está no `.gitignore` e o repositório só tem o `.env.example`. A varredura de chaves (`sk_`, `pk_`, `whsec_` e tokens do GitHub) em todo o histórico e na árvore de trabalho, feita em 13/11/2026, não encontrou nenhuma ocorrência.

3.7 Observabilidade, CI/CD e documentação da API

Health checks e logs. Cada serviço expõe `/health`; o gateway agrega o estado de todos em `/health`. Os eventos de negócio saem em uma linha JSON por evento no log do processo (`apps/api` e `billing-service`; exemplos em `docs/evidencias/logs-eventos-de-negocio.txt`) e na tabela de auditoria. **Métricas** (Prometheus ou similar) e identificador de correlação entre serviços não foram feitos; estão no backlog (issue #7).

Testes. Há 43 testes no billing, 29 no auth-tenant, 101 no gateway, 364 na api (66 pulados que dependem do worker R) e 274 no front, todos passando na execução de 13/11/2026 (`docs/evidencias/testes.txt`).

CI/CD. Não há pipeline rodando no GitHub. O workflow do GitHub Actions foi escrito (`docs/ci/ci.yml`, testes de cada serviço e do front), mas o push dele foi recusado porque o token de acesso não tem o escopo `workflow`; por isso ele ainda não está em `.github/workflows/` e nunca foi executado. **Documentação da API:** o FastAPI gera o OpenAPI de cada serviço automaticamente; não preparei uma página de Swagger unificada.

3.8 Camadas IaaS, PaaS e SaaS do projeto

Camada	O que é no meu projeto	Quem gerencia
IaaS	A máquina virtual, a rede e o disco onde a stack do Portainer roda, e a segunda máquina que hospeda o worker R.	o provedor da VM; eu administro o sistema e o Docker dentro dela
PaaS	Docker e Portainer (build a partir do repositório, variáveis, volumes, reinício), SQLite em volumes e o Garage (objetos S3) que subimos como contêiner. Não há PostgreSQL gerenciado nem Redis: é uma lacuna (issue #6).	eu, sobre a VM
SaaS	O Metastatistic para o cliente final (landing, app, planos) e os SaaS que eu consumo: Stripe (pagamento, assinatura, portal), GitHub (repositório, issues, entrega) e o Portainer como painel.	eu (o produto); Stripe e GitHub (fornecedores)

4 Quadro de entregas

Nº	Atividade	Data planejada	Data realizada	Situação
7	Plano Premium — cobrança de verdade com Stripe	09/10/2026	09/10/2026 20:29	entregue
8	SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve	16/10/2026	—	não entregue
9	Fork e plano de negócio	23/10/2026	10/11/2026 10:15	entregue com atraso
10	Seu negócio no ar — o domínio no lugar dos filmes	06/11/2026	13/11/2026 12:53	entregue com atraso
11	SaaS completo — planos, limites e assinatura com Stripe	13/11/2026	13/11/2026 12:53	entregue

Fontes: a atividade 7 pela issue #7 do repositório do catálogo; da 9 em diante, pelas issues de entrega do repositório do produto. Pela regra da avaliação, a atividade 8 (não entregue) e as atividades 9 e 10 (entregues com atraso) descontam 0,25 cada da P2.

5 Atividades realizadas

5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe

Descrição	Plano pago em modo de teste: Checkout Session do Stripe e webhook marcando o usuário como premium.
Data planejada	09/10/2026
Data realizada	09/10/2026 20:29
Situação	entregue
Evidência	Issue #7 do repositório do catálogo (menciona o professor) e commit com a Checkout Session e o webhook
Link	https://github.com/Felipe-Takayuki/atividade-filmes/issues/7

O que foi feito. O catálogo de filmes ganhou um plano Premium com o Stripe em modo de teste. O clique em “Seja Premium” cria uma Checkout Session e redireciona para a página hospedada do Stripe, de modo que o catálogo nunca vê dados de cartão (o que também tira o sistema do escopo do PCI-DSS). A ativação do plano vem de um webhook assinado (`POST /api/stripe/webhook`), verificado criptograficamente, e não da volta do navegador. O plano Free passou a limitar os favoritos a 5 e o Premium libera favoritos ilimitados e mostra um selo nos comentários. A issue descreve o fluxo e o guia de testes, e há uma suíte automatizada cobrindo webhook, limites e conformidade. O trabalho foi commitado em 25/09/2026 (commits `c8b66e8` a `e60dd92`); a issue de entrega é de 09/10/2026 20:29, que é o horário usado como realizado.

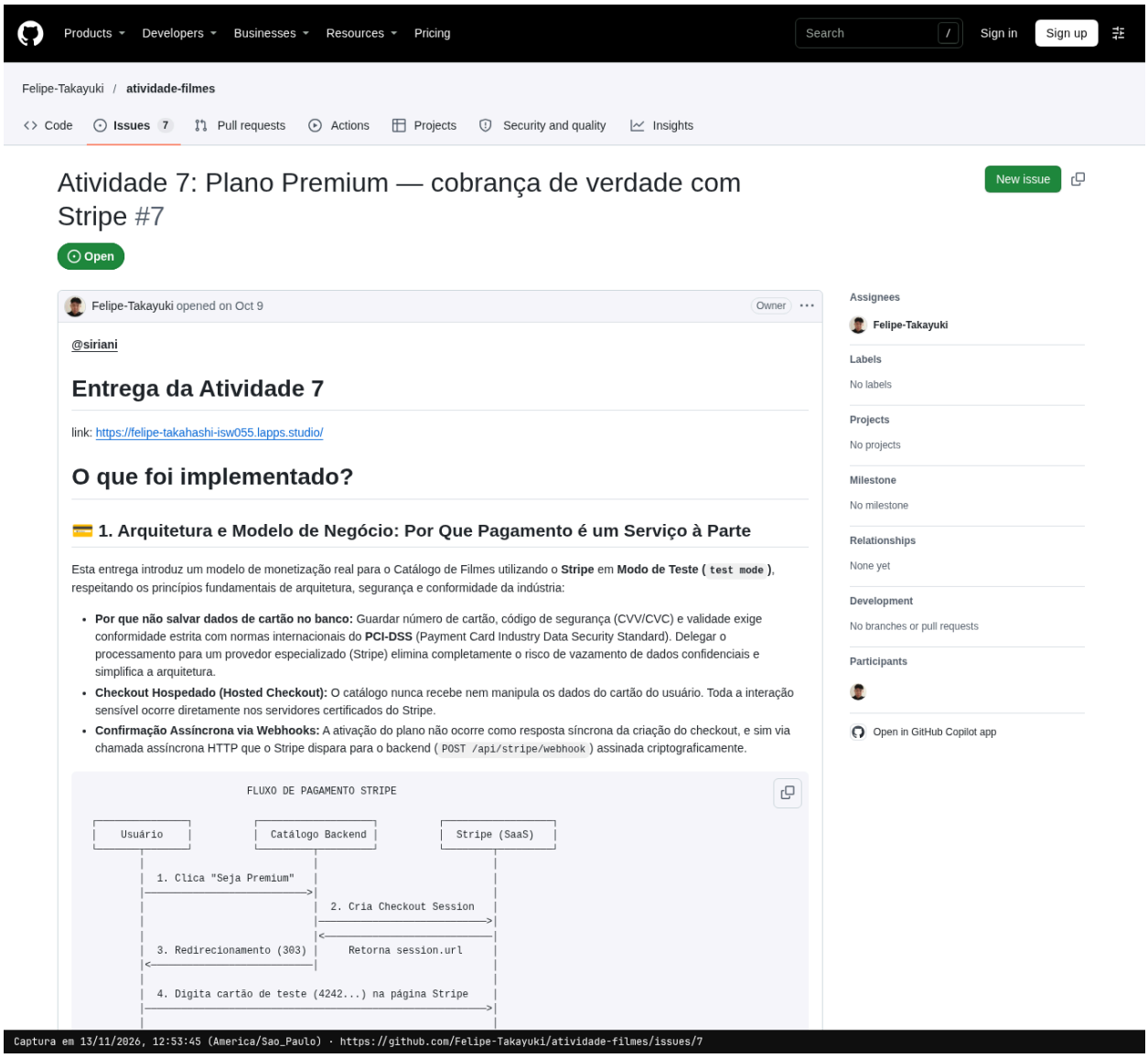


Figura 4: Atividade 7 — issue de entrega, com o professor mencionado (captura em 13/11/2026 12:53)

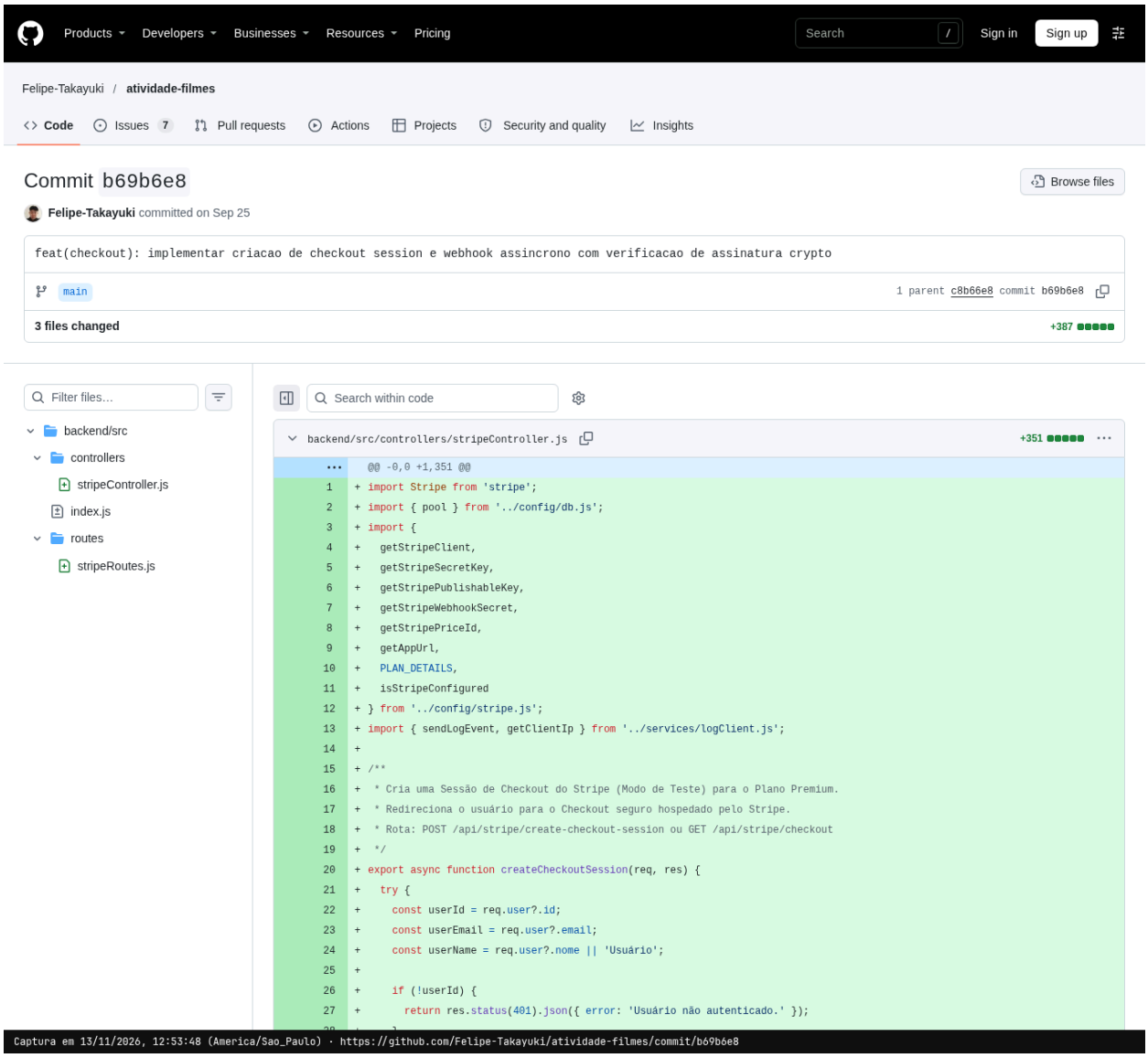


Figura 5: Atividade 7 — commit com a Checkout Session e o webhook (captura em 13/11/2026 12:53)

Observação sobre a evidência. Não anexo print do sistema funcionando: o endereço publicado na issue respondeu 502 em 13/11/2026, e o resultado se apoia no commit, na issue e nos testes.

5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve

Descrição	Camadas de nuvem do próprio projeto e documentação reversa: especificação, arquitetura, regras de negócio e modelo de dados.
Data planejada	16/10/2026
Data realizada	—
Situação	não entregue
Evidência	Não há evidência: não existe issue de entrega nem DOCUMENTACAO.md nos repositórios
Link	—

O que faltou e por quê. Não entreguei esta atividade no prazo. Procurei nos dois repositórios (o do catálogo e o do produto), nas issues e na árvore de arquivos, e não existe issue de entrega nem o arquivo

`DOCUMENTACAO.md` pedido. Não tenho registro de um impedimento técnico: a atividade simplesmente não foi feita no prazo. O que existe agora é o conteúdo equivalente escrito para o produto final, na seção de arquitetura deste relatório (componentes, fluxos, modelo de dados e as camadas IaaS, PaaS e SaaS) e no `README` e no `PLAN.md` do repositório do produto. Isso não substitui a entrega e não conta como tal: é só o registro do que existe hoje.

5.3 Atividade 9 — Fork e plano de negócio

Descrição	Repositório novo com o histórico do anterior, README do produto (dor, persona, Lean Canvas, modelo de receita, mapa de reaproveitamento) e backlog em issues.
Data planejada	23/10/2026
Data realizada	10/11/2026 10:15
Situação	entregue com atraso
Evidência	Issue #1 do repositório do produto (menciona o professor), README do produto e histórico de commits
Link	https://github.com/Felipe-Takayuki/metastatistic-commercial/issues/1

O que foi feito. O produto ganhou repositório próprio, `metastatistic-commercial`, e um README com o plano de negócio: nome e frase, a dor (quem sofre, quando, quanto custa e como resolve hoje), a persona (Dra. Camila Valente, coordenadora de pesquisa clínica), concorrentes (SPSS e SAS, Power BI e Metabase, planilhas, JASP e RStudio), proposta de valor, modelo de receita com Stripe, métricas, o Lean Canvas e o mapa de reaproveitamento da arquitetura do curso (login e papéis, logs, upload, Stripe, observabilidade). O backlog está em issues: as issues #2 a #8 são o que falta (partição por organização, Stripe real, cota de armazenamento, identidade única, PostgreSQL, observabilidade e reconciliação de webhooks) e a #1 é a entrega.

A entrega foi com atraso: o prazo era 23/10 e a issue de entrega foi aberta em 10/11/2026 às 10:15. O README do commit de 09/11 tinha, no modelo de receita, preços que depois mudaram: em 13/11 o README foi atualizado para os planos Free e Pro em reais, e o backlog, que antes só aparecia como texto, passou a existir como issues reais.

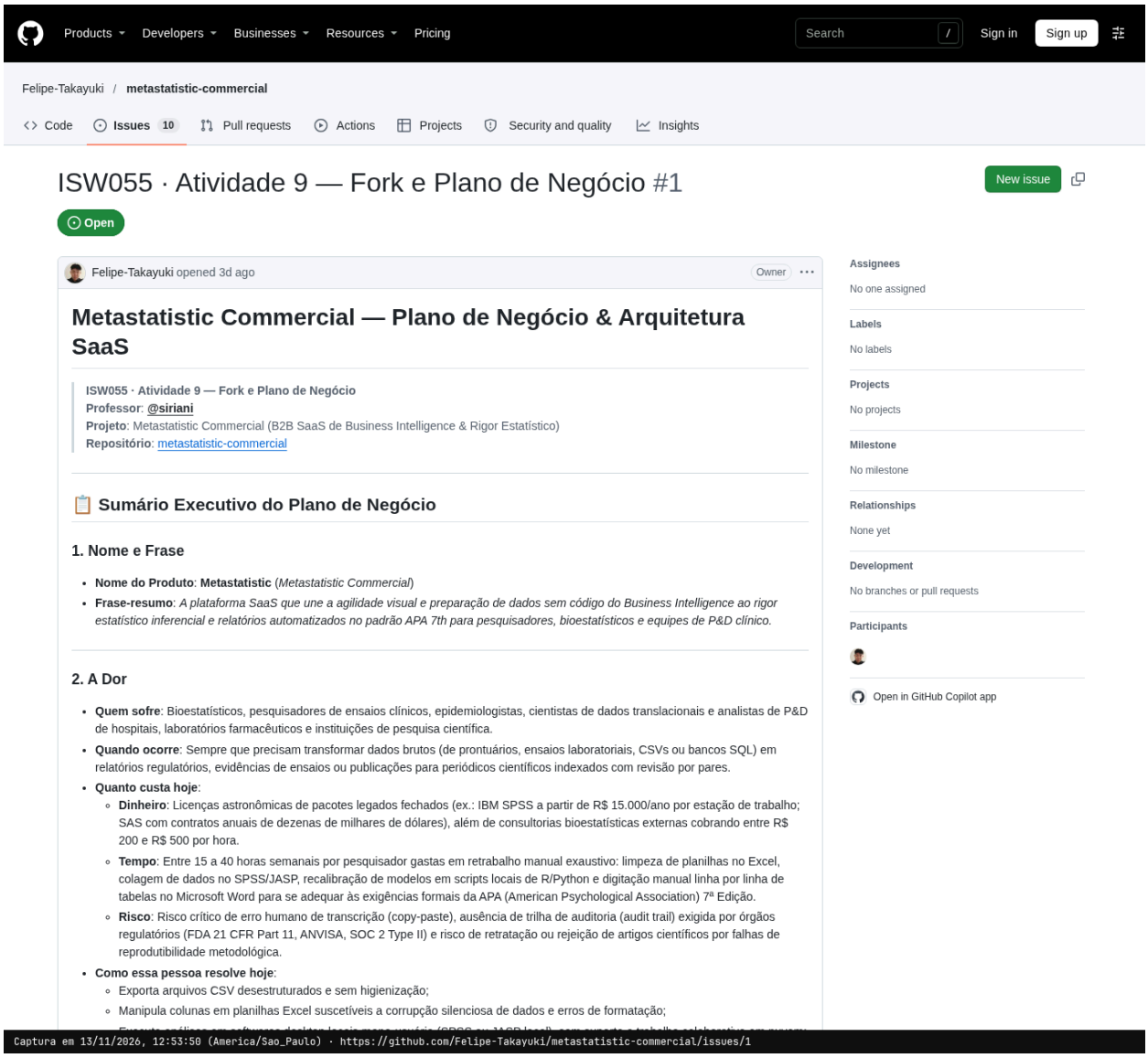


Figura 6: Atividade 9 — issue de entrega #1, com o professor mencionado (captura em 13/11/2026 12:53)

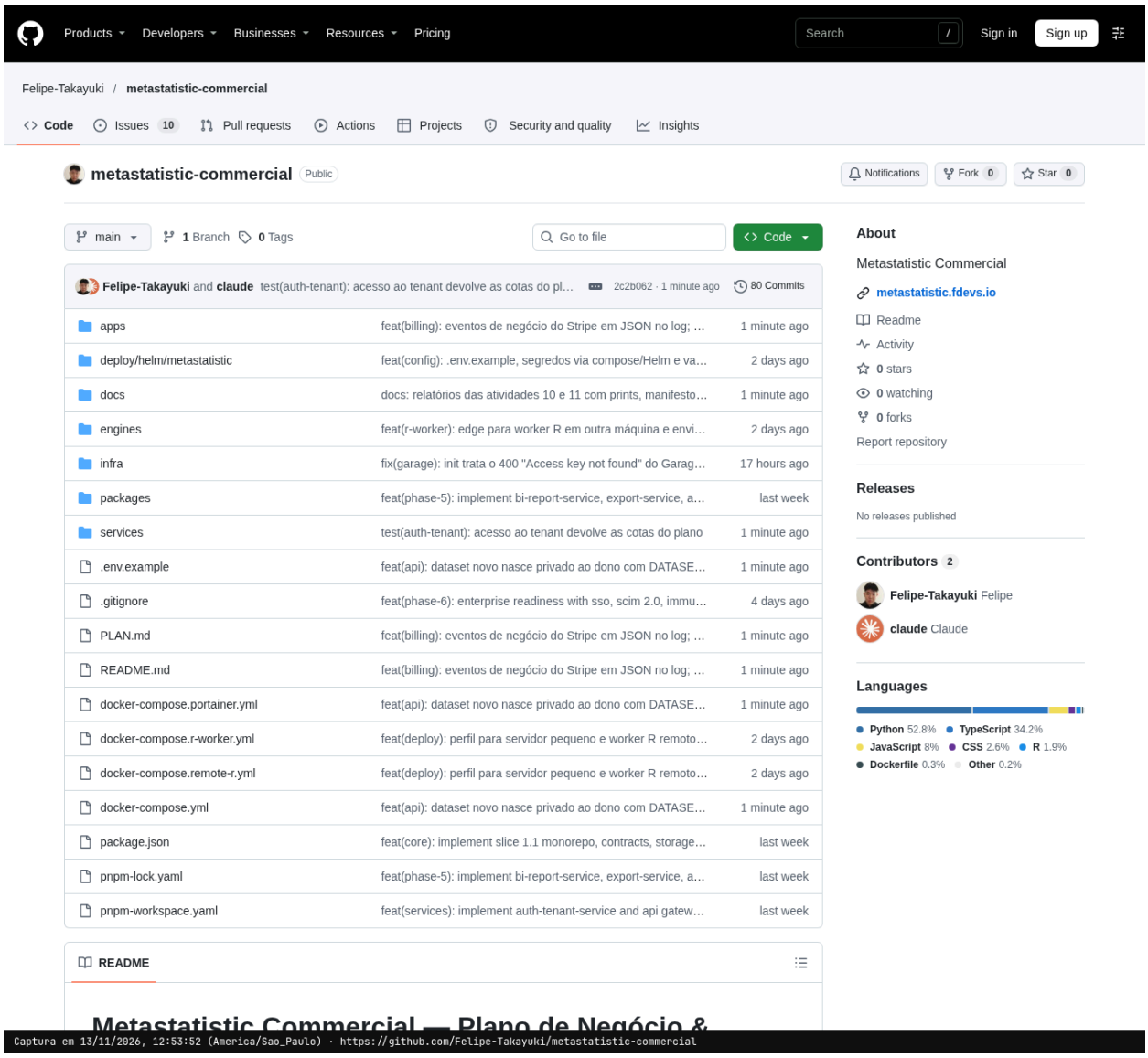


Figura 7: Atividade 9 — README do produto e lista de issues do backlog (captura em 13/11/2026 12:53)

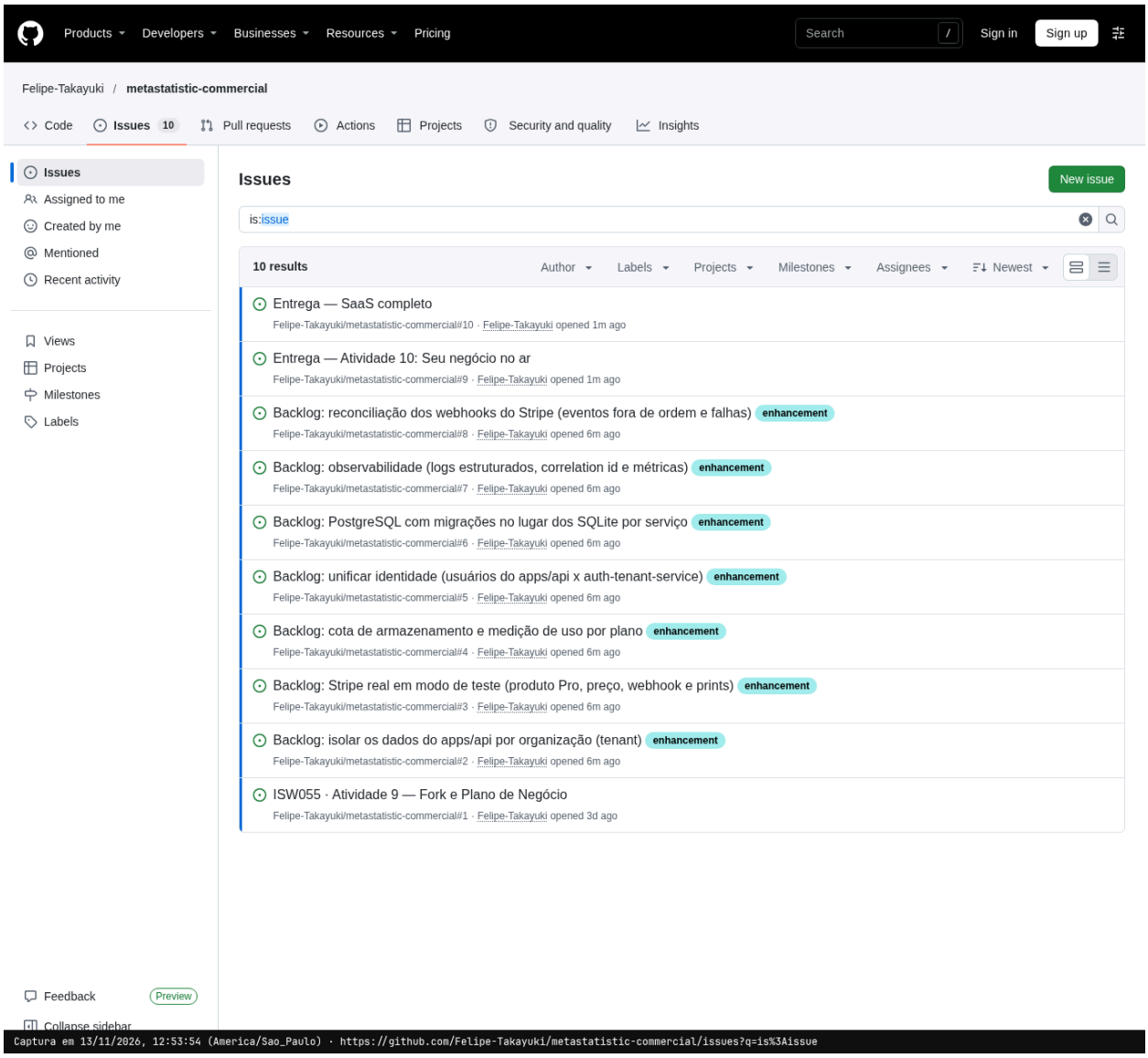


Figura 8: Atividade 9 — issues do repositório (captura em 13/11/2026 12:53)

Dificuldades e como foram resolvidas. O plano e o código divergiam em preço e moeda (R$ no plano, US$ no código). Isso só foi resolvido na atividade 11, com um catálogo único (Free e Pro, em reais, só mensal) refletido no README, no `PLAN.md`, na landing e no billing.

5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes

Descrição	Stack apontando para o repositório novo, tabelas e API do domínio, papéis do negócio, isolamento por usuário, eventos no log, upload e landing page.
Data planejada	06/11/2026
Data realizada	13/11/2026 12:53
Situação	entregue com atraso
Evidência	Issue #9 (menciona o professor), relatório docs/atividade-10-negocio-no-ar.md, prints com data e hora e log de eventos
Link	https://github.com/Felipe-Takayuki/metastatistic-commercial/issues/9

O que foi feito. O domínio saiu de filmes e virou datasets, análises, relatórios e organizações: tabelas e API do negócio, papéis da plataforma (`viewer`, `editor`, `admin`) e da organização, upload de CSV de dados de estudo (vira Parquet com metadados e respeita o plano), landing page pública com os planos, e eventos de negócio tanto na tabela de auditoria quanto em linhas JSON no log do processo. Cada usuário vê só os próprios dados: o dataset nasce privado ao dono e outro usuário não o enxerga (404). A entrega teve atraso: o prazo era 06/11, e a issue de entrega foi aberta em 13/11/2026 às 12:53, depois de eu corrigir o que faltava para o requisito de isolamento e de eventos no log valer de fato.

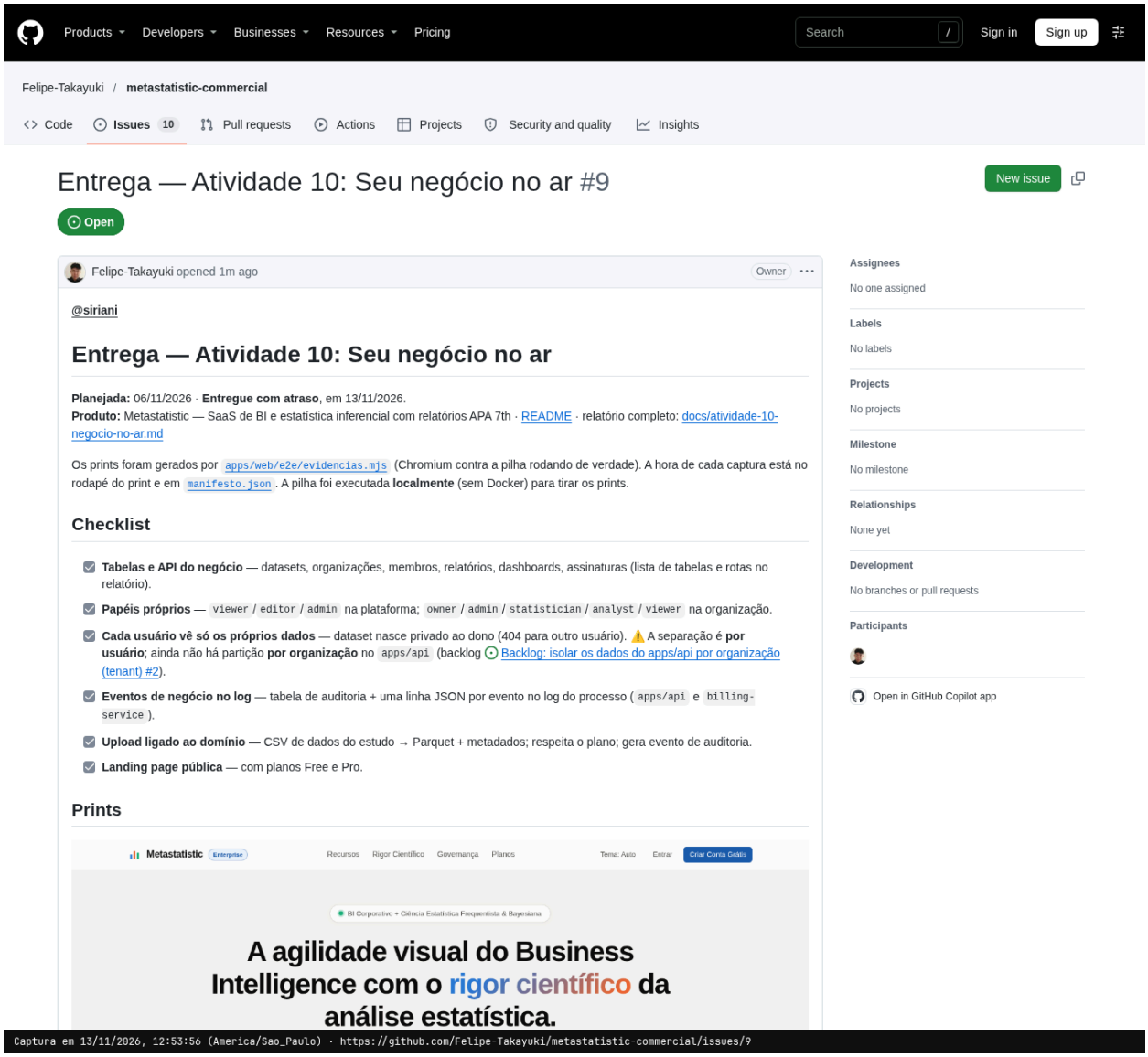


Figura 9: Atividade 10 — issue de entrega #9 (captura em 13/11/2026 12:53)

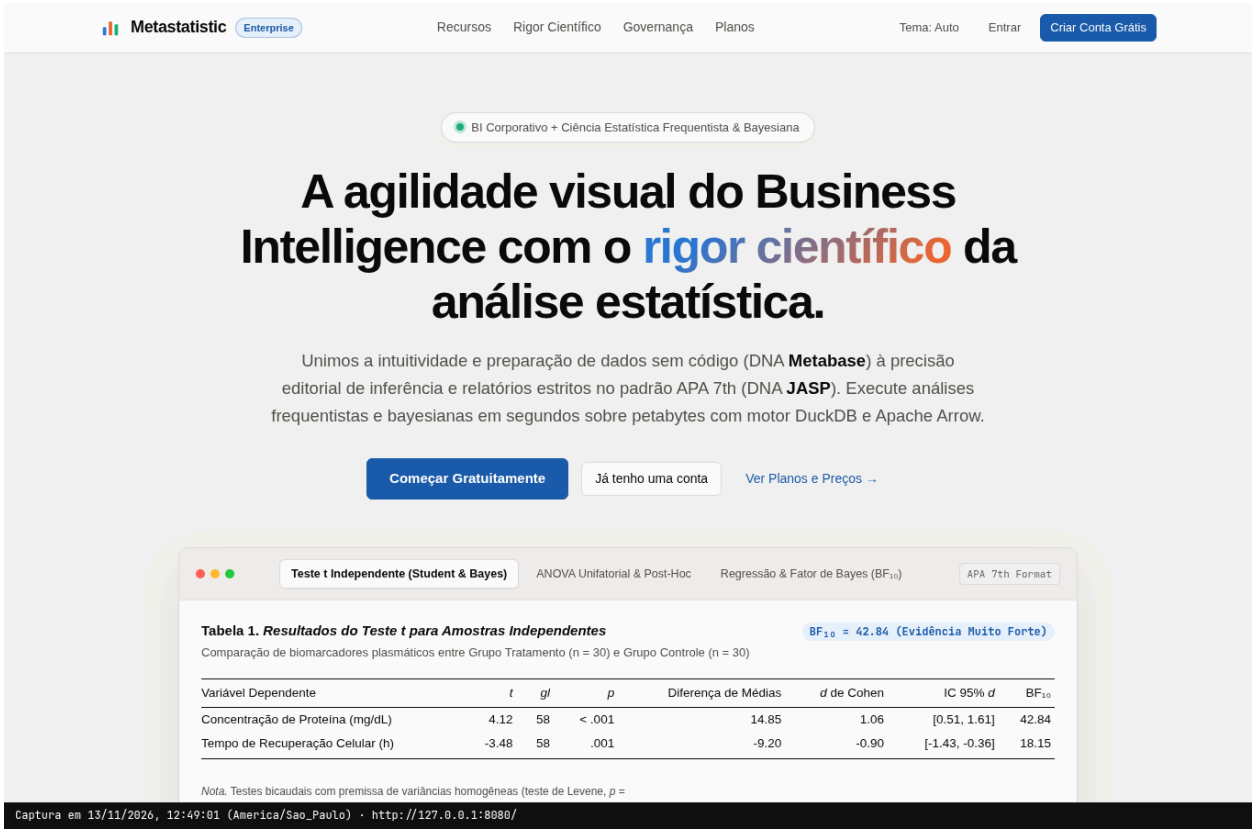


Figura 10: Atividade 10 — landing page pública (captura em 13/11/2026 12:49)

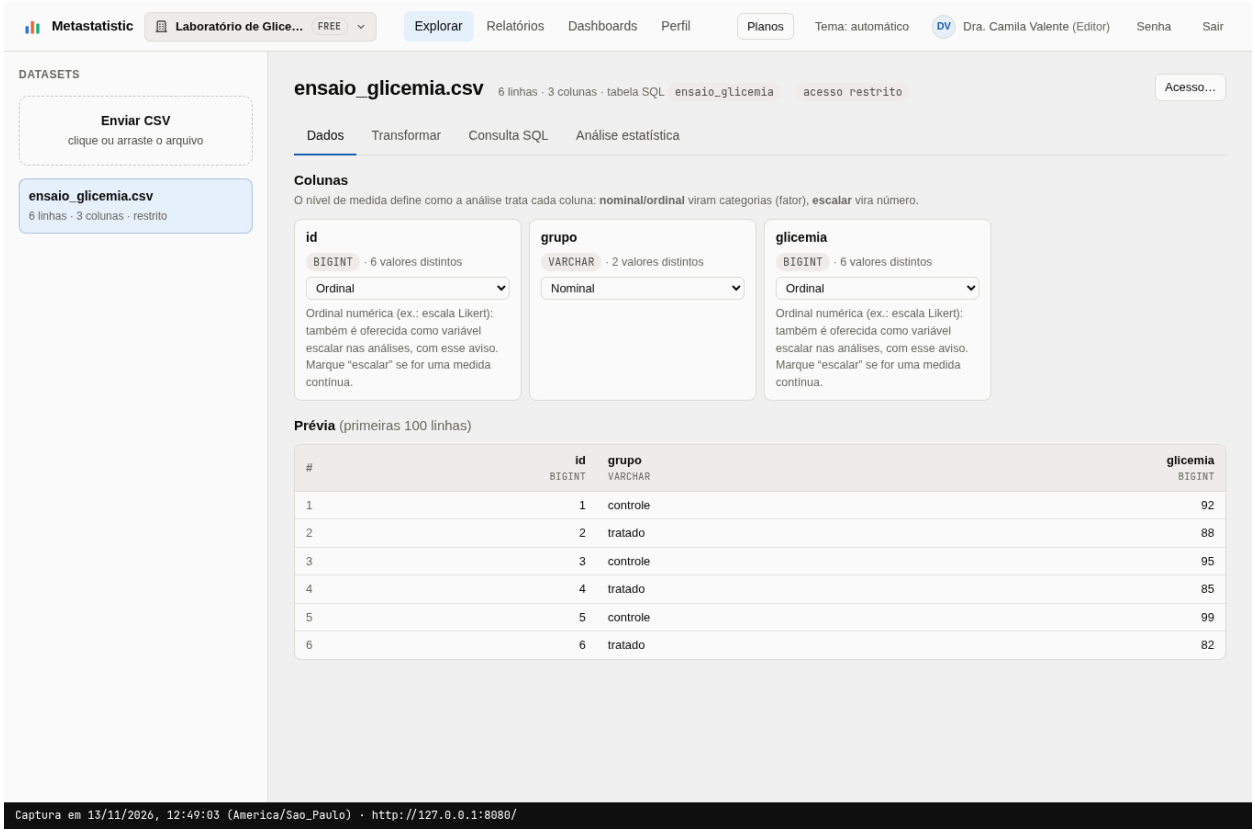


Figura 11: Atividade 10 — a Camila vê o dataset que enviou (captura em 13/11/2026 12:49)

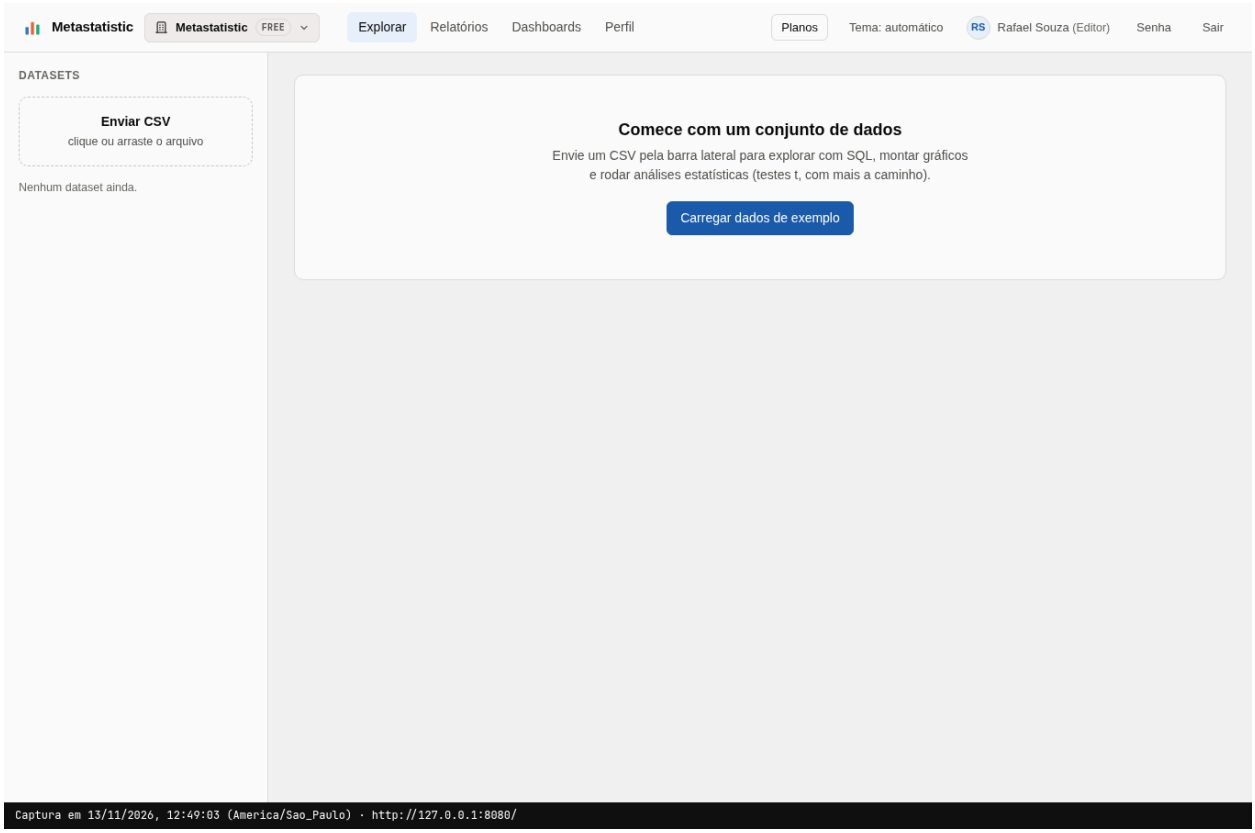


Figura 12: Atividade 10 — o Rafael, outro usuário, não vê o dataset dela (captura em 13/11/2026 12:49)

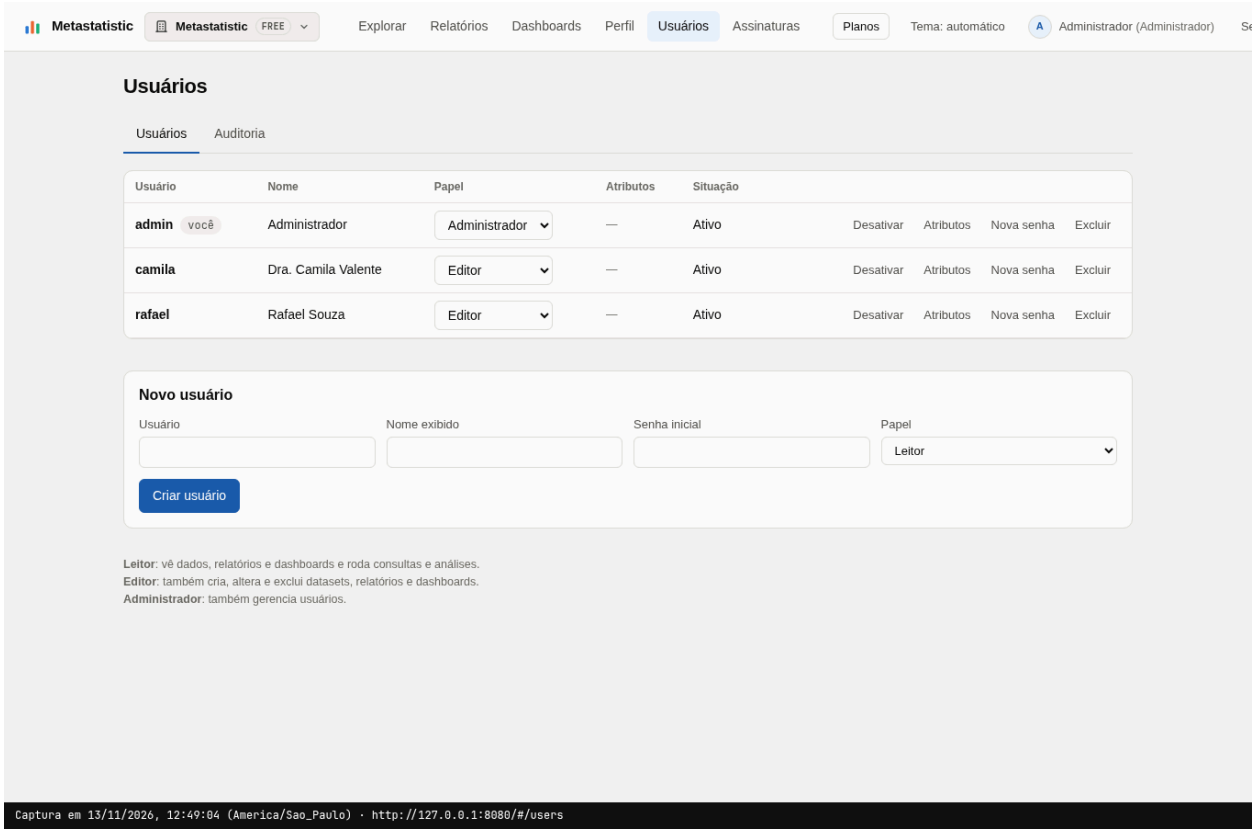


Figura 13: Atividade 10 — papéis dos usuários (captura em 13/11/2026 12:49)

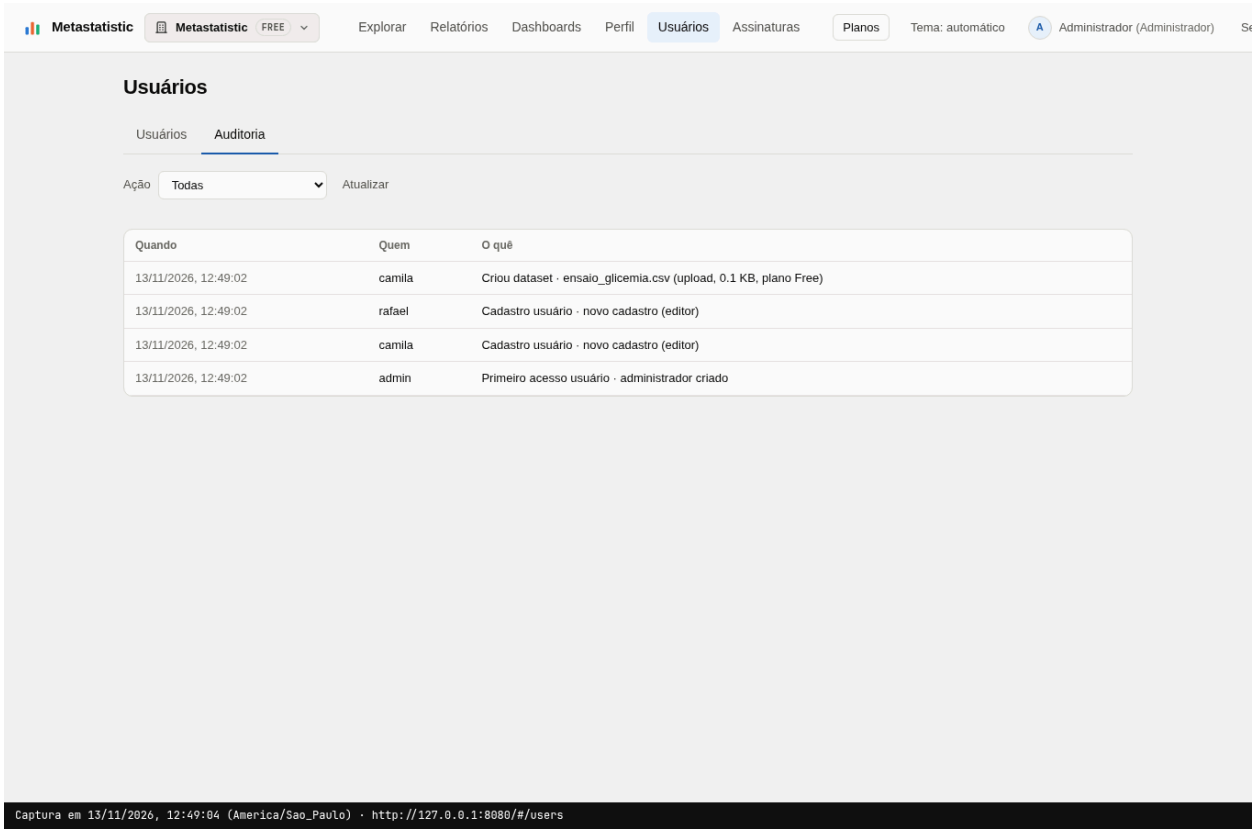


Figura 14: Atividade 10 — eventos de negócio na auditoria (captura em 13/11/2026 12:49)

Dificuldades e como foram resolvidas. Ao verificar o requisito do isolamento, o padrão era "dataset aberto a todos que entraram"; passei a criar o dataset privado ao dono (variável `DATASETS_PRIVATE_BY_DEFAULT`, ligada no compose). Continua pendente a separação por organização (issue #2). Os prints foram feitos com a pilha rodando localmente, não em um endereço público.

5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe

Descrição	Planos recorrentes, checkout de assinatura, webhooks atualizando o plano, portal do cliente, limites por plano na API e painel admin.
Data planejada	13/11/2026
Data realizada	13/11/2026 12:53
Situação	entregue
Evidência	Issue #10 (menciona o professor), relatório docs/atividade-11-saas-stripe.md, prints do plano, do limite bloqueando e do painel admin (Stripe simulado)
Link	https://github.com/Felipe-Takayuki/metastatistic-commercial/issues/10

O que foi feito. Planos Free (R$ 0) e Pro (R$ 249,00 por mês, assinatura recorrente mensal em reais); checkout de assinatura e portal do cliente ("Gerenciar assinatura"); webhooks validados por assinatura e idempotentes pelo `event.id` para `checkout.session.completed`, `customer.subscription.updated`, `customer.subscription.deleted` e `invoice.payment_failed`; limites por plano aplicados na API (datasets, tamanho do arquivo, usuários), com mensagem que diz o limite e como liberar; e um painel admin com usuários por plano, assinaturas ativas, MRR de teste e últimos eventos. O gateway recusa criar organização já em plano pago (antes dava para criar uma Pro sem pagar). Nenhuma chave está no repositório (varredura do histórico inteiro: zero ocorrências).

Limitação importante. Os prints abaixo foram feitos com o Stripe **simulado** (`STRIPE_MOCK_MODE=true`): o checkout e os webhooks foram simulados, e o painel admin mostra "Stripe simulado (mock)". O caminho do Stripe real existe no código (SDK e verificação de `Stripe-Signature`), mas não foi exercitado contra o Stripe em modo de teste, e por isso não tenho aqui print do checkout com o cartão 4242, da página do Stripe nem do portal. Isso está registrado como pendência na issue #3, com o passo a passo.

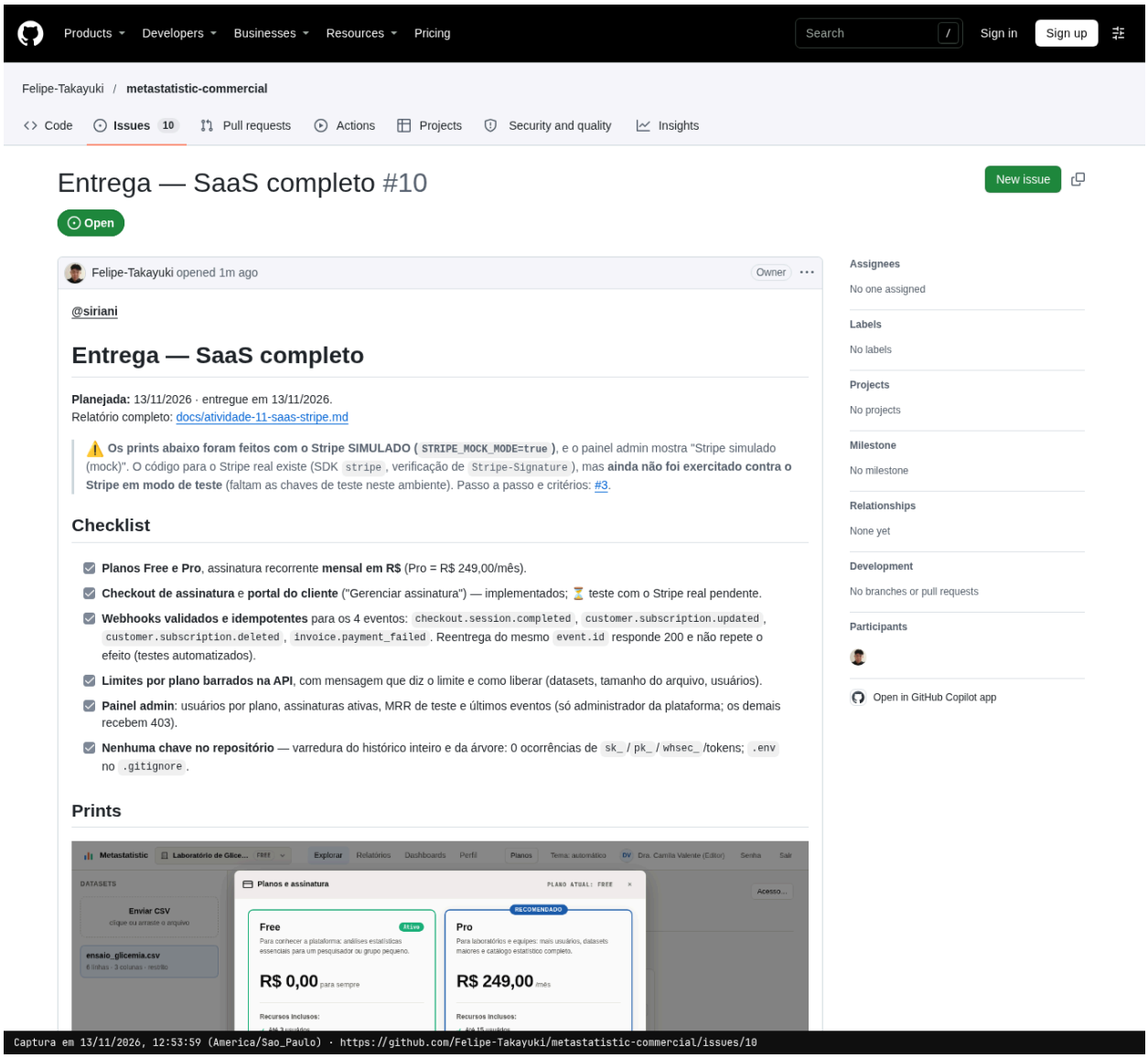


Figura 15: Atividade 11 — issue de entrega #10 (captura em 13/11/2026 12:53)

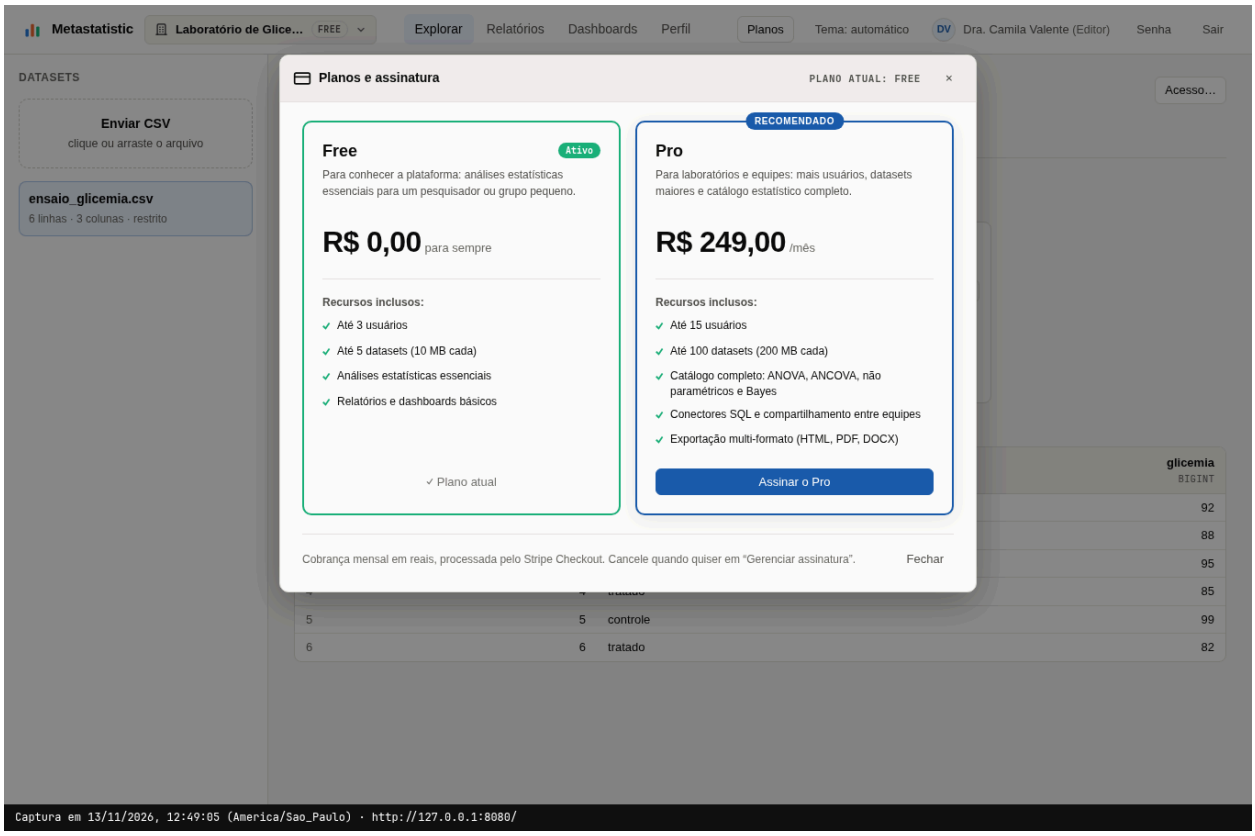


Figura 16: Atividade 11 — planos Free e Pro (captura em 13/11/2026 12:49)

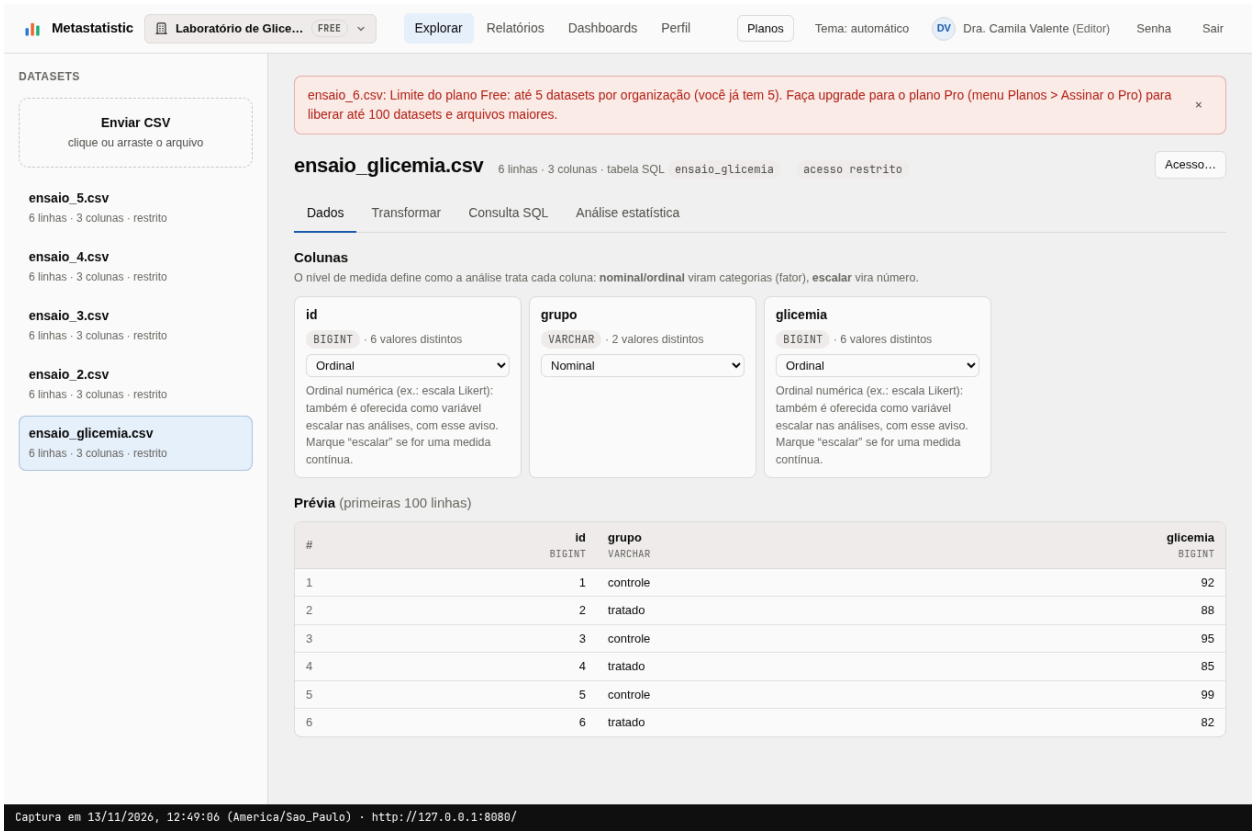


Figura 17: Atividade 11 — o sexto dataset é barrado pela API no Free, com o limite e como liberar (captura em 13/11/2026 12:49)

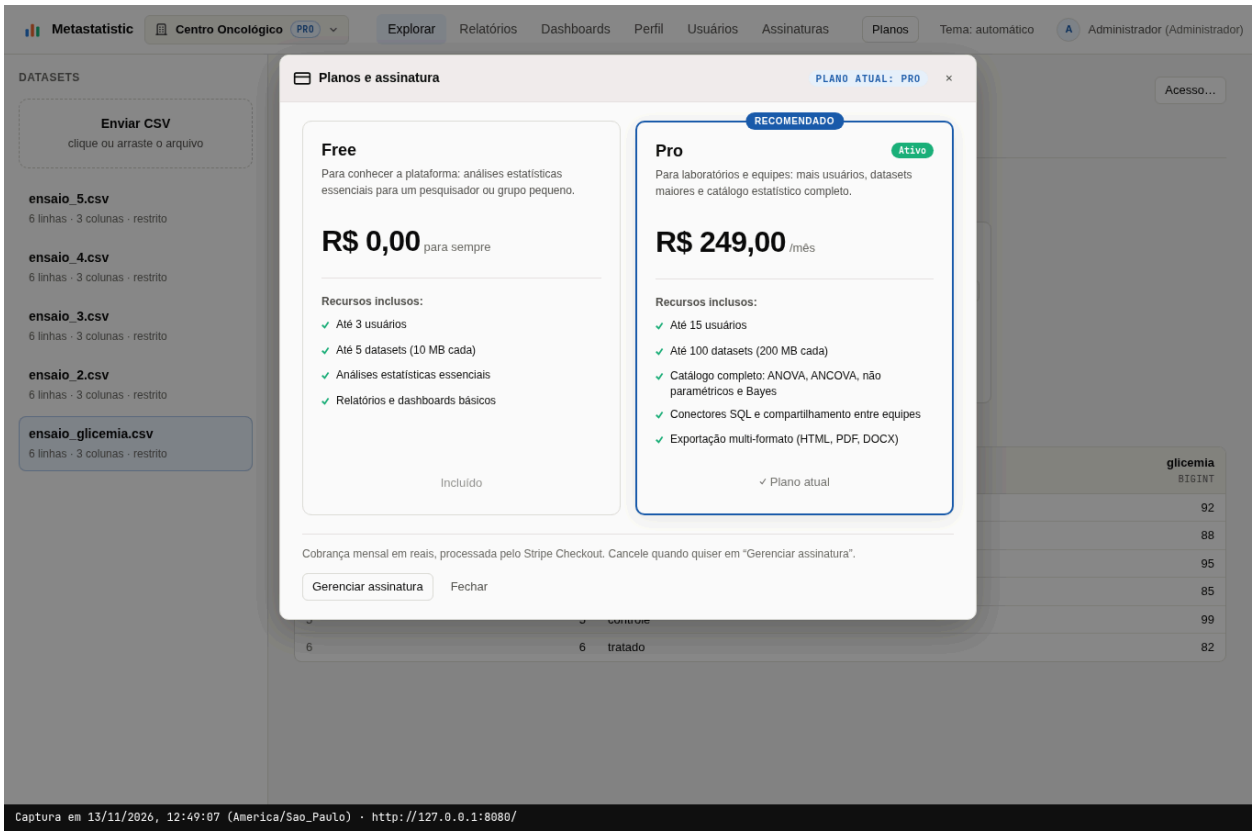


Figura 18: Atividade 11 — organização com o Pro ativo (webhook simulado; captura em 13/11/2026 12:49)

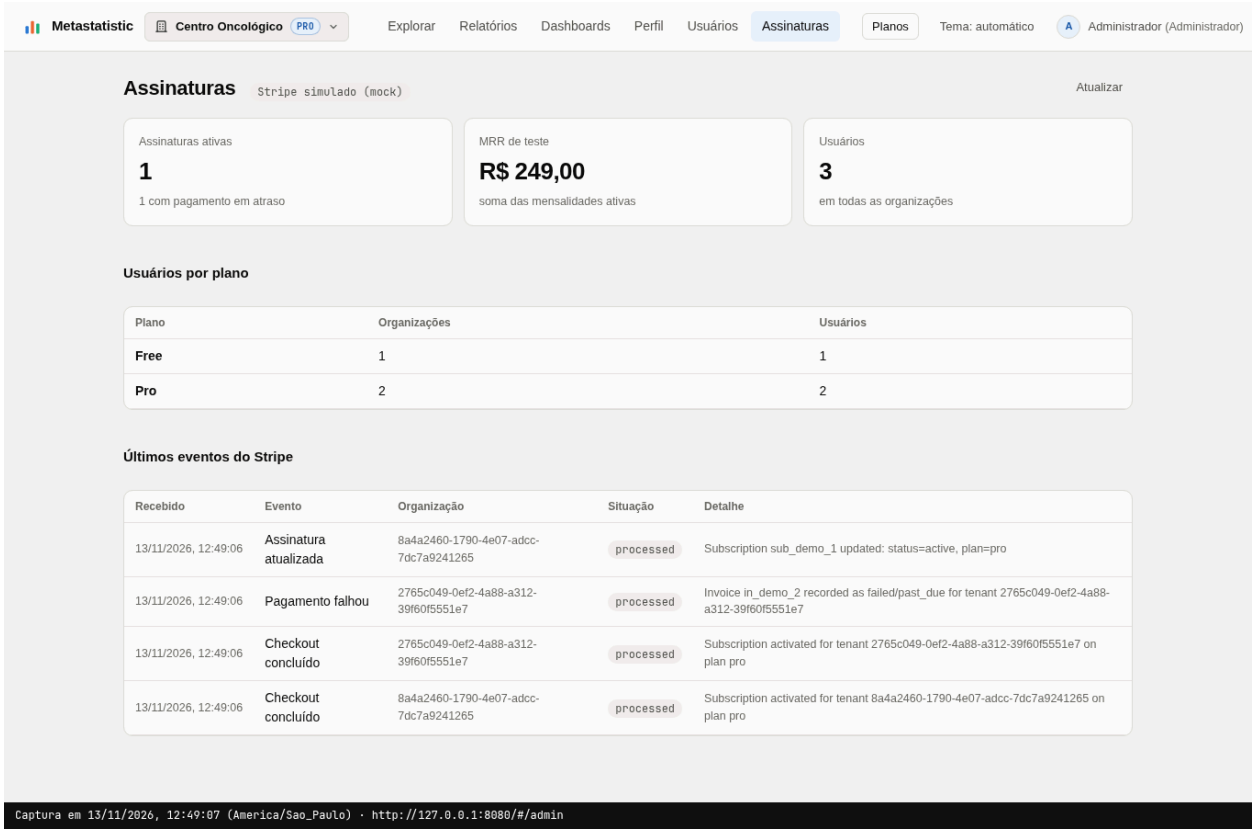


Figura 19: Atividade 11 — painel admin (Stripe simulado; captura em 13/11/2026 12:49)

Dificuldades e como foram resolvidas. Três achados: (1) o modal de criar organização permitia escolher Team ou Enterprise sem pagar, corrigido no gateway; (2) o gateway repassava o cabeçalho `Content-`

`Encoding` depois de descomprimir o corpo, o que quebrava o navegador quando o servidor comprimia; (3) o plano precisava chegar até a API sem poder ser forjado, resolvido com cabeçalhos que o gateway descarta do cliente e preenche com o que o auth-tenant confirmou. A reentrega do mesmo evento foi testada: responde 200, `duplicate: true` e não repete o efeito.

6 Considerações finais

O que mais me ensinou neste bimestre foi que "funcionar" e "estar pronto para vender" são coisas diferentes. Os conceitos de nuvem (serviços separados, contêineres, gateway, webhooks, camadas IaaS, PaaS e SaaS) apareceram de verdade quando precisei decidir onde um limite de plano é aplicado, o que acontece se o Stripe entrega um evento duas vezes e como impedir que o cliente forje o próprio plano. O mais difícil foi o prazo: três atividades saíram atrasadas e uma não foi entregue, e fica registrado aqui sem atenuante.

Faria diferente em duas coisas. Primeiro, tratar a cobrança como parte do produto desde o começo: preço, moeda e plano divergiram por semanas entre o plano de negócio e o código. Segundo, testar o Stripe real antes de montar as evidências: ficou a lacuna de não ter o checkout e o portal reais registrados.

Para o Metastatistic ser um produto de verdade ainda faltam: o teste ponta a ponta com o Stripe em modo de teste (issue #3), isolar os dados por organização (#2), unificar o cadastro de usuários da plataforma e das organizações (#5), trocar os SQLite por PostgreSQL com migrações (#6), observabilidade com métricas e identificador de correlação (#7), reconciliar webhooks fora de ordem (#8) e publicar o pipeline de CI. A atividade 8 continua em aberto.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados.

Felipe Takayuki Tanaka Takahashi

Pompeia, 13/11/2026